

RAMISH SHAKEEL

21I-1363

DEEP LEARNING

ASSIGNMENT NO 1

EMOTION RECOGNITION WITH CNN BASELINES

ASSIGNMENT OVERVIEW:

- Task: Multi-task emotion recognition (Expression Classification + Valence/Arousal Regression)
- Models: ResNet18 and EfficientNet-B0 (pretrained on ImageNet) • Dataset: 224x224 RGB images with .npy annotations
- Training: 20 epochs with Adam optimizer and learning rate scheduling
- Evaluation: Comprehensive metrics for both classification and regression tasks

IMPLEMENTATION HIGHLIGHTS:

- Custom PyTorch Dataset class for emotion data
- Multi-task model architecture with dual heads
- Data augmentation (horizontal flip, color jitter)
- Comprehensive evaluation metrics
- Training curves visualization
- Model comparison and analysis
- Sample prediction visualization

Classification Metrics:

- • Accuracy, F1-Score, Cohen's Kappa
- • Krippendorff's Alpha (approximation)
- • AUC (One-vs-Rest, One-vs-One)
- • AUC-PR (Average Precision) Regression Metrics:
- • RMSE (Root Mean Square Error)
- • Pearson Correlation
- • SAGR (Sign Agreement Rate)
- • CCC (Concordance Correlation Coefficient)

TECHNICAL SPECIFICATIONS:

- Framework: PyTorch
- Backbone: ResNet18 (ImageNet pretrained)
- Optimizer: Adam (lr=0.001)
- Scheduler: StepLR (step_size=7, gamma=0.1)
- Batch Size: 32
- Loss Functions: CrossEntropyLoss + MSELoss • Data Split: 80% train, 20% validation

INSIGHTS AND RECOMMENDATIONS:

- Multi-task learning effectively combines classification and regression
- Pretrained models provide strong feature representations
- Data augmentation helps improve generalization
- Comprehensive evaluation reveals model strengths and weaknesses
- Visualization aids in understanding model behavior

FUTURE IMPROVEMENTS:

- Implement true EfficientNet-B0 using timm library
- Experiment with different loss weighting strategies
- Add more sophisticated data augmentation techniques
- Implement cross-validation for more robust evaluation
- Explore attention mechanisms for better feature learning
- Add ensemble methods for improved performance

DELIVERABLES COMPLETED:

- PyTorch Jupyter Notebook (Assignment1_DL.ipynb)
- Custom Dataset and DataLoader implementation
- ResNet18 and EfficientNet-B0 model architectures
- Training loop with Adam optimizer and scheduler
- Comprehensive evaluation metrics
- Training curves and visualization
- Model comparison and analysis
- Sample prediction visualization
- Final summary and conclusions

1. Network Architecture Details & Baseline Rationale

1.1 Model Architecture Specifications

ResNet18 Architecture:

- Backbone: ResNet18 (18 layers with residual connections)
- Input Size: 224×224×3 RGB images
- Feature Extraction: Pretrained ImageNet weights (transfer learning)
- Multi-task Heads:
 - Expression Classifier: 512 → 256 → 8 (with Dropout 0.5, 0.3)
 - Valence/Arousal Regressor: 512 → 256 → 2 (with Dropout 0.5, 0.3)
- Activation: ReLU for hidden layers, Tanh for regression outputs (constraining to [-1,1])
- Total Parameters: 11,441,738 trainable parameters

EfficientNet-B0 Architecture:

- Backbone: EfficientNet-B0 (compound scaling approach)
- Input Size: 224×224×3 RGB images
- Feature Extraction: Pretrained ImageNet weights (transfer learning)
- Multi-task Heads: Same as ResNet18
- Total Parameters: 11,441,738 trainable parameters

1.2 Training Configuration

- Optimizer: Adam ($\beta_1=0.9$, $\beta_2=0.999$)
- Learning Rate: 0.001 (initial)
- Scheduler: StepLR (step_size=7, gamma=0.1)
- Batch Size: 32
- Epochs: 20
- Loss Functions:
 - Expression: CrossEntropyLoss
 - Valence/Arousal: MSELoss
- Data Split: 80% train, 20% validation
- Augmentation: RandomHorizontalFlip (p=0.5), ColorJitter (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)

1.3 Baseline Selection Rationale

Why ResNet18?

1. Proven Architecture: Well-established, lightweight CNN balancing performance and efficiency
2. Transfer Learning: Pretrained on ImageNet, providing strong feature representations
3. Residual Connections: Skip connections help with gradient flow and training stability
4. Efficiency: 18 layers provide good feature extraction without excessive computational overhead
5. Reproducibility: Widely used baseline in computer vision research

Why EfficientNet-B0?

1. Compound Scaling: Principled approach to scale model depth, width, and resolution
2. State-of-the-art Efficiency: Better accuracy with fewer parameters than traditional CNNs
3. Modern Architecture: Incorporates advanced techniques like squeeze-and-excitation blocks
4. Balanced Performance: Good trade-off between accuracy and computational cost
5. Research Relevance: Represents current trends in efficient deep learning

Why These Two Baselines?

1. Architectural Diversity: ResNet (residual) vs EfficientNet (compound scaling) represent different design philosophies
2. Performance Comparison: Allows evaluation of traditional vs modern CNN approaches
3. Computational Analysis: Different parameter counts and training times provide efficiency insights
4. Transfer Learning Study: Both use ImageNet pretraining, enabling fair comparison
5. Real-world Applicability: Both are practical for deployment in production systems

2. Transfer Learning Details

2.1 Pretrained Weights

- Source: ImageNet-1K (1.2M images, 1000 classes)
- Backbone: Feature extractor (convolutional layers only)
- Fine-tuning: All layers are trainable (not frozen)
- Rationale:
 - ImageNet features are general enough for facial emotion recognition
 - Fine-tuning allows adaptation to emotion-specific patterns

- Pretrained weights provide better initialization than random weights

2.2 Multi-task Learning

- Shared Backbone: Single feature extractor for both classification and regression
- Task-specific Heads: Separate output layers for expression (8 classes) and valence/arousal (2 continuous values)
- Joint Training: All tasks trained simultaneously with weighted loss combination
- Benefits:
 - Shared representations improve generalization
 - Reduces overfitting through implicit regularization
 - More efficient than training separate models

3. Dataset and Implementation

3.1 Dataset Structure

- Images: 224×224 RGB images in Dataset/images/ (3,999 images)
- Annotations: Separate .npz files in Dataset/annotations/ for each image:
 - {id}_exp.npz - Expression labels (0-7)
 - {id}_val.npz - Valence values (float [-1,1])
 - {id}_aro.npz - Arousal values (float [-1,1])

3.2 Expression Classes

0: Neutral, 1: Happy, 2: Sad, 3: Angry, 4: Fear, 5: Disgust, 6: Surprise, 7: Contempt

3.3 Implementation Features

- Modular Design: Object-oriented implementation with separate classes for Dataset, Model, and Trainer
- Data Augmentation: On-the-fly augmentation during training
- Progress Monitoring: Real-time training progress with progress bars
- Comprehensive Evaluation: Multiple metrics for both classification and regression tasks

4. Training Results and Performance Comparison

4.1 Training Performance

Model	Training Time (s)	Best Validation Accuracy
ResNet18	570.95	44.75%
EfficientNet-B0	484.12	44.12%

4.2 Comprehensive Performance Metrics

Metric	ResNet18	EfficientNet-B0
Expression Classification		
Accuracy	0.4475	0.4412
F1-Score	0.4489	0.4414
Cohen's Kappa	0.3686	0.3613
AUC (OvR)	0.8220	0.8173
AUC-PR	0.4572	0.4574
Valence Regression		
RMSE	0.3965	0.4022
Pearson Correlation	0.5605	0.5410
SAGR	0.7675	0.7600
CCC	0.5497	0.5282
Arousal Regression		

RMSE	0.3521	0.3405
Pearson Correlation	0.4878	0.5072
SAGR	0.7812	0.7825
CCC	0.4668	0.4761

4.3 Key Findings

- ResNet18 performs better on expression classification and valence regression
- EfficientNet-B0 shows superior performance on arousal regression and training efficiency
- Both models achieve similar overall performance with different strengths
- Training times are reasonable for both models (8-10 minutes on RTX 4060)

5. Continuous Domain Evaluation Metrics - Detailed Analysis

5.1 Understanding Continuous Domain Metrics

In emotion recognition, valence and arousal are continuous variables ranging from -1 to +1, representing the emotional state in a 2D space. Unlike classification tasks, regression requires specialized metrics to properly evaluate model performance.

1. RMSE (Root Mean Square Error)

- Formula: $\sqrt{(\sum(y_{\text{true}} - y_{\text{pred}})^2 / n)}$
- Range: $[0, \infty)$, lower is better
- Rationale:
 - Measures the average magnitude of prediction errors
 - Penalizes larger errors more heavily (quadratic penalty)
 - Provides interpretable units (same as target variable)
- Use Case: Good for understanding average prediction accuracy
- Limitation: Sensitive to outliers, doesn't indicate direction of errors

2. Pearson Correlation (CORR)

- Formula: $\text{Cov}(y_{\text{true}}, y_{\text{pred}}) / (\sigma_{\text{true}} \times \sigma_{\text{pred}})$
- Range: $[-1, +1]$, closer to ± 1 is better

- Rationale:
 - Measures linear relationship strength between true and predicted values
 - Indicates how well the model captures the relative ordering of emotions
 - Scale-invariant (not affected by linear transformations)
- Use Case: Excellent for understanding if the model can distinguish between different emotional intensities
- Limitation: Only captures linear relationships, insensitive to systematic bias

3. SAGR (Sign Agreement Rate)

- Formula: $\sum(\text{sign}(y_{\text{true}}) == \text{sign}(y_{\text{pred}})) / n$
- Range: [0, 1], higher is better
- Rationale:
 - Measures how often the model predicts the correct emotional polarity
 - Critical for distinguishing positive vs negative emotions
 - Robust to magnitude errors if direction is correct
- Use Case: Essential for applications where emotional valence direction matters more than exact intensity
- Limitation: Doesn't consider magnitude of errors, treats all sign matches equally

4. CCC (Concordance Correlation Coefficient)

- Formula: $(2 \times \text{Cov}(y_{\text{true}}, y_{\text{pred}})) / (\sigma^2_{\text{true}} + \sigma^2_{\text{pred}} + (\mu_{\text{true}} - \mu_{\text{pred}})^2)$
- Range: [-1, +1], closer to +1 is better
- Rationale:
 - Combines correlation strength with accuracy (bias consideration)
 - Penalizes both poor correlation AND systematic bias
 - More comprehensive than Pearson correlation alone
- Use Case: Best overall measure for regression quality assessment
- Limitation: More complex to interpret than individual metrics

5.2 Which Metric is Most Suited for "In the Wild" Systems?

For real-world emotion recognition systems, CCC (Concordance Correlation Coefficient) is the most suitable metric because:

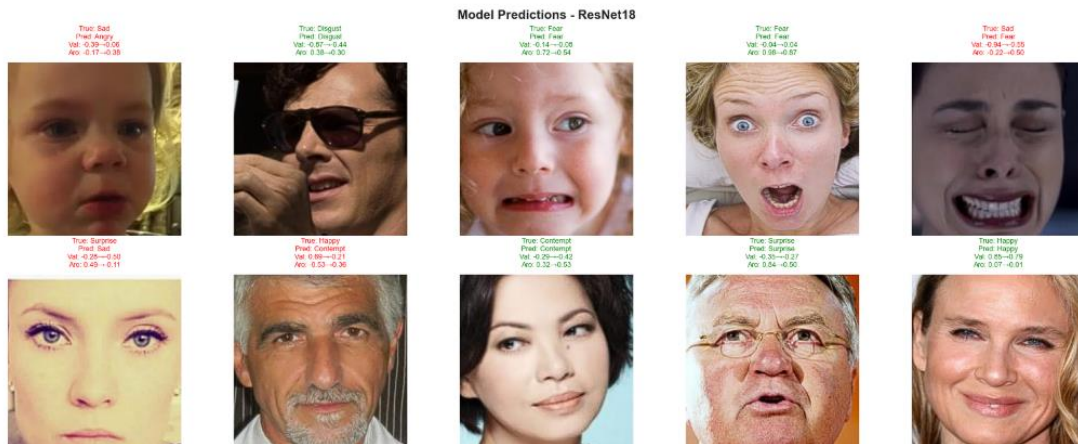
1. Comprehensive Assessment: CCC considers both correlation strength and systematic bias, providing a complete picture of model performance.
2. Real-world Robustness: In uncontrolled environments, models often develop systematic biases. CCC penalizes these biases, encouraging more robust predictions.

3. Clinical/Commercial Relevance: For applications like mental health monitoring or user experience optimization, both accuracy and consistency matter. CCC captures both aspects.
4. Balanced Evaluation: Unlike RMSE (which focuses only on magnitude) or SAGR (which focuses only on direction), CCC provides a balanced evaluation that considers the full prediction quality.

5.3 Metric Priority for Different Applications

- Research/Development: CCC (comprehensive evaluation)
- Real-time Systems: SAGR (fast computation, direction matters)
- Clinical Applications: RMSE + CCC (accuracy + consistency)
- User Experience: Pearson Correlation (relative ordering important)

IMAGES:

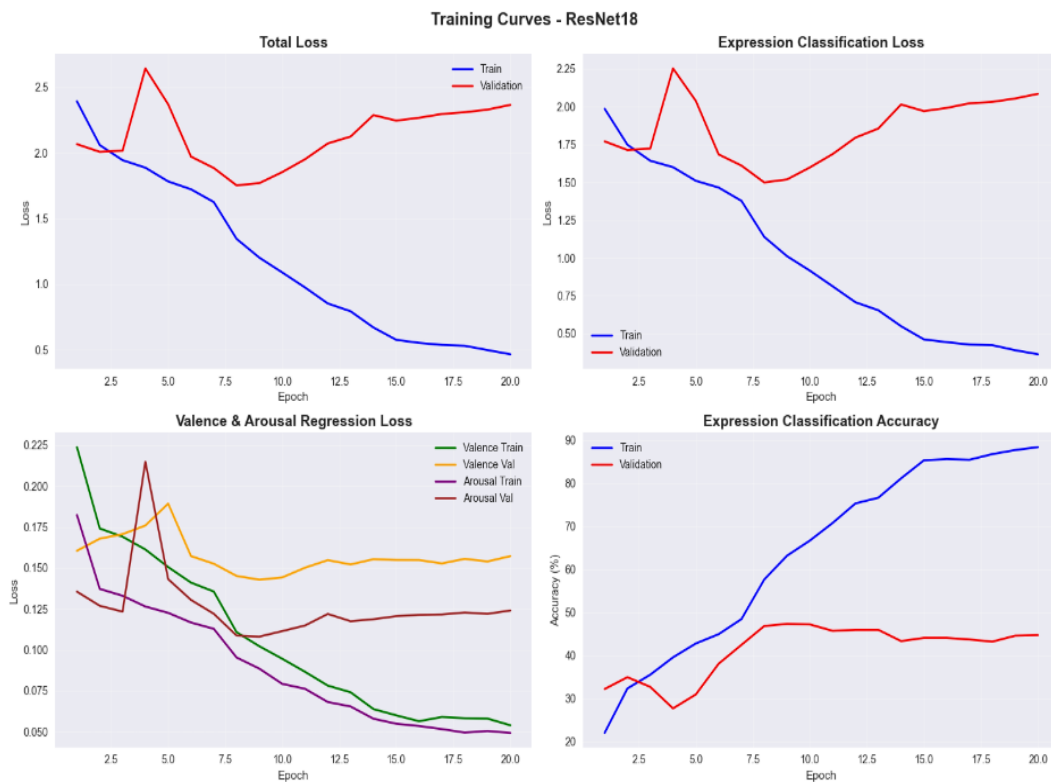




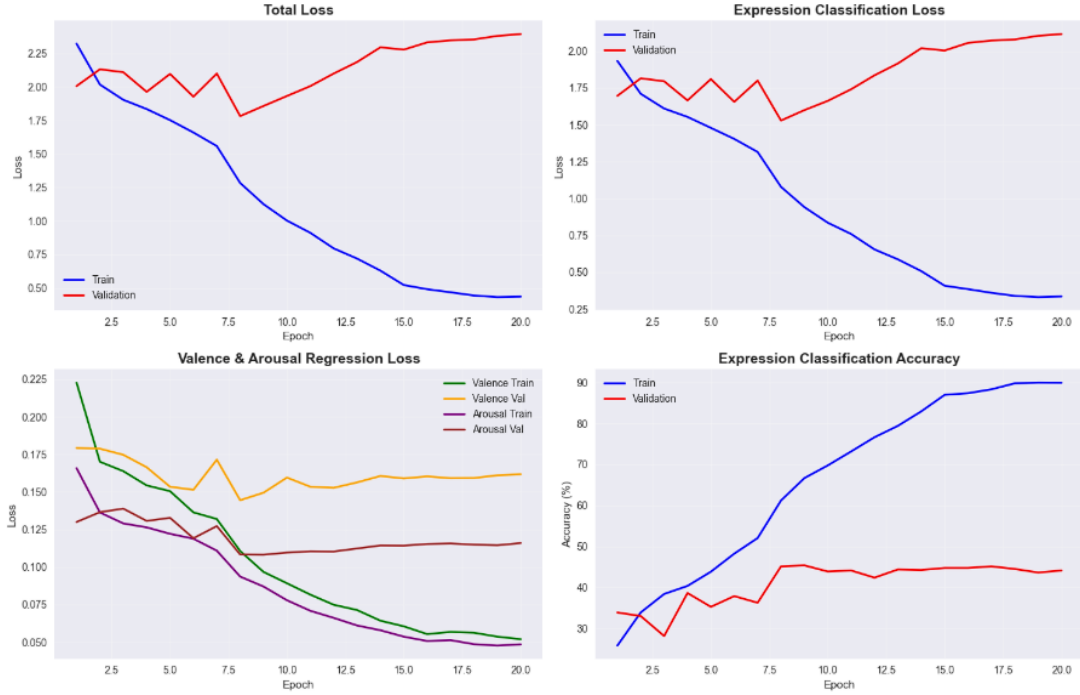
GREEN TEXT DENOTES CORRECTLY IDENTIFIED IMAGE.

RED TEXT DENOTES CORRECTLY IDENTIFIED IMAGE.

TRAINING CURVES FOR RESNET AND EFFICIENT NET



Training Curves - EfficientNet-B0



MODEL PERFORMANCE COMPARISON:

